I3C Controller IP – Design Specification

Version: 1.0

Overview

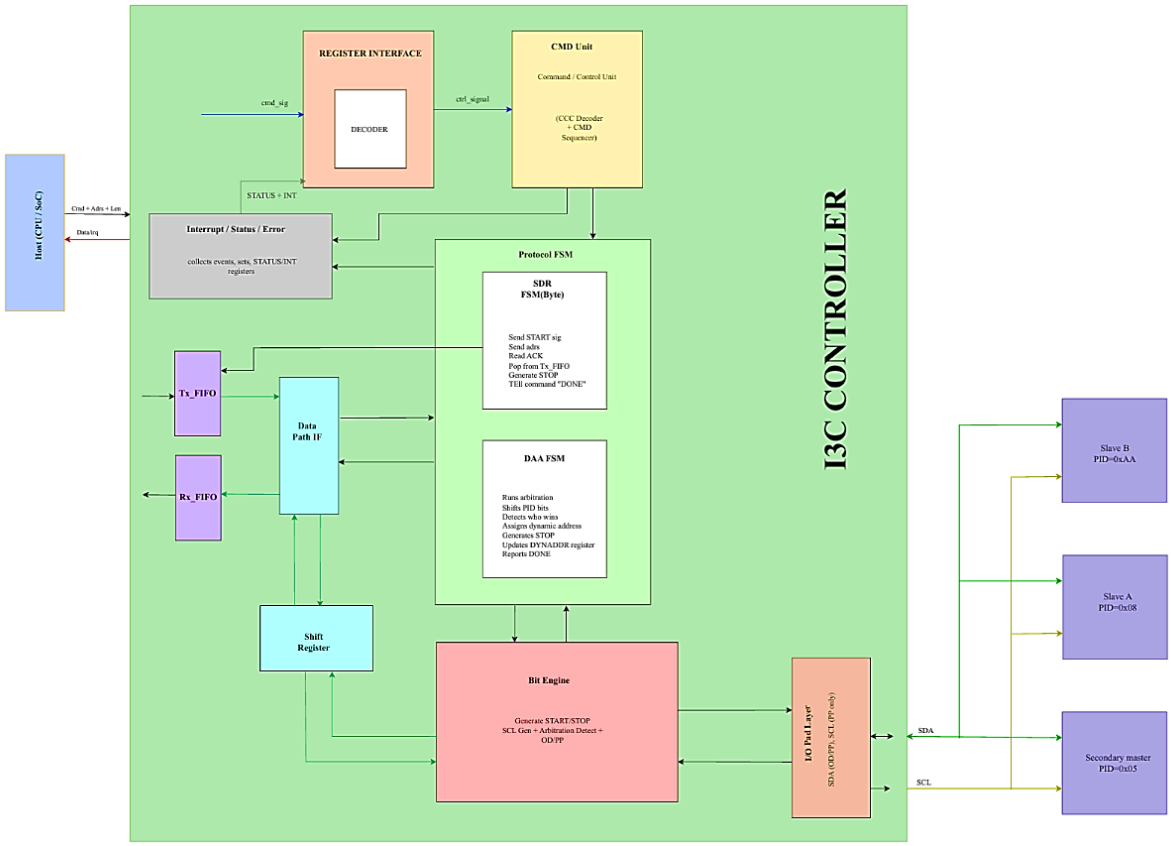
This document describes the architecture and behavior of an I3C Controller IP (SDR-only) implemented as a synthesizable RTL design for FPGA/ASIC validation.

The controller acts as an Active Controller (Master) on an I3C Basic bus and supports:

- SDR private transfers (read/write)
- Broadcast and Direct CCCs (limited)
- Dynamic Address Assignment (ENTDAA) using open-drain arbitration
- Legacy I²C compatibility for non-I3C targets

The design is structured as layered engines, separating protocol sequencing, byte/bit transfer, and physical bus control.

Block Diagram



Scope of Version 1

Version 1 includes:

- SDR Mode (Single Data Rate)

- Push-Pull & Open-Drain signaling
- START, STOP, Repeated START
- 7-bit Dynamic Addressing
- Private Read/Write transfers
- 128-byte TX/RX FIFO
- DAA (ENTDAA)
- Basic CCC: ENTDAA, GETSTATUS, GET PID , BCR , DCR
- Register interface (APB/AXI-Lite)
- Basic error reporting

Version 1 does **not** include:

- HDR Modes
- Hot-Join
- In-Band Interrupts (IBI)
- Multi-controller arbitration
- Advanced CCCs
- Timing control commands
- Low-power extensions

1. SCOPE AND FEATURES

The following table summarizes features included and excluded in Version 1.0:

Feature	V1 Status	Notes
SDR Mode (Single Data Rate)	Included	Up to 12.5 MHz, controller-driven SCL.
Push-Pull & Open-Drain	Included	Open-drain for START/STOP/ACK/DAA; push-pull for data/T-bit.
START / STOP / Repeated START	Included	Implemented in SDR FSM.
Private Read / Write	Included	Byte-oriented transfers via TX/RX FIFOs.
Dynamic Address Assignment (ENTDAA)	Included	DAA FSM assigns dynamic addresses and updates device table.
Basic CCCs (ENTDAA,GET STATUS)	Included	Executed by CMD FSM using SDR transfers.
Generic Register Interface	Included	Simple reg_cs/reg_wr/reg_addr/reg_wdata/reg_rdata/reg_ready.
TX/RX FIFOs (128-byte)	Included	Synchronous FIFOs for buffering.
IRQ output	Included (optional)	Asserts on DONE/ERROR when enabled.
APB/AXI-Lite Interface	Excluded	May be added externally as a wrapper.
HDR Modes	Excluded	SDR-only in V1.
Hot-Join	Excluded	Not implemented in V1.
In-Band Interrupts (IBI)	Excluded	Not implemented in V1.
Multi-controller support	Excluded	Single main controller only.

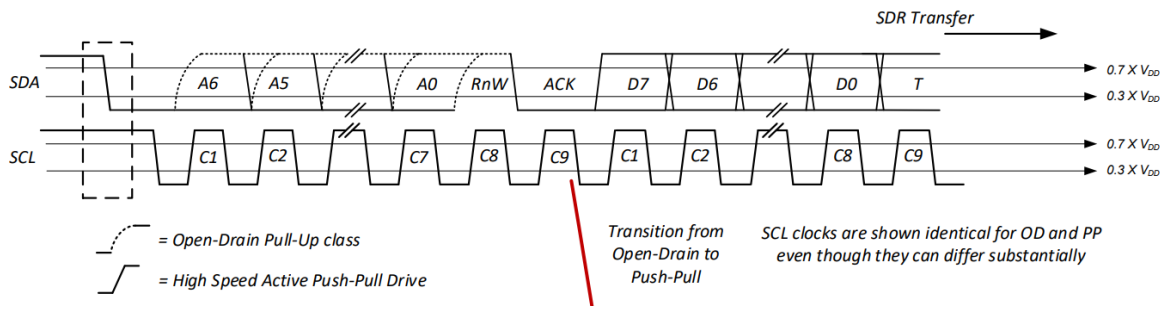
Power management features	Excluded	No sleep/wake extensions in V1.

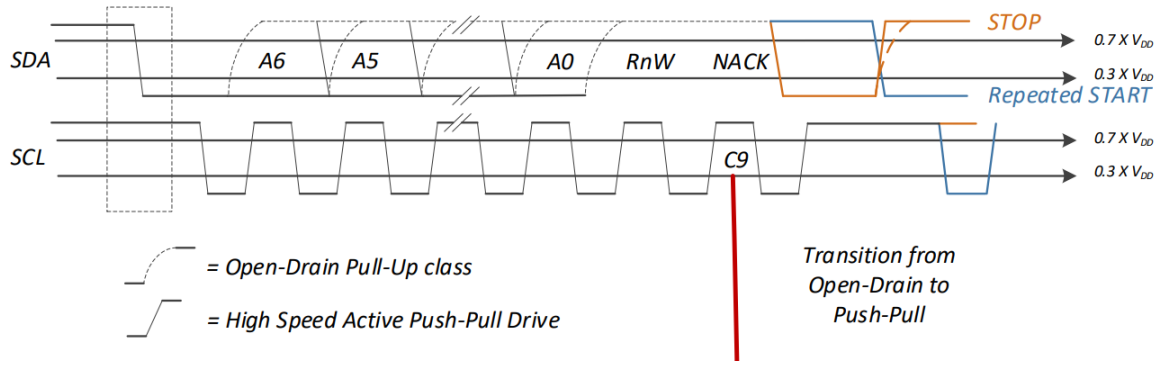
2. DETAILED DESCRIPTION

2.1 SDR Mode (Single Data Rate)

- SDR supports transfer speeds up to **12.5 MHz** using push-pull signaling.
- SCL is generated entirely by the Controller.
- SDA is sampled once per SCL rising edge.

2.2 Dual Signaling Modes: Open-Drain + Push-Pull





Open-Drain Mode

Used for:

- **ALL THE I2C TRANSFERS** (ADDR, DATA both are sent in open drain for I2C devices)
- START
- STOP
- ACK/NACK
- Arbitration during ENTDAA

Controller only pulls SDA low; logic-high is via pull-ups.

Push-Pull Mode

Used for:

- Address header

- Data bytes
- T-bit(ACK/NACK for now)
- CCC payload

Faster and lower power.

2.3 Private Write (Controller → Target)

- Controller sends: START → Address+W → ACK → Data bytes → STOP
- Each data byte is followed by target ACK.
- Up to **128 bytes** per SDR transaction.
- NACK → transfer terminated + status update.

2.4 Private Read (Target → Controller)

- Controller sends: START → Address+R → ACK
- Target drives data (push-pull)
- Controller ACKs until last byte
- Controller issues NACK → STOP
- T-bit handling included

2.5 FIFO Buffers (128-Byte TX & RX)

- TX FIFO holds outgoing write data
- RX FIFO holds incoming read data
- Depth chosen per SDR transaction limit (128B)

2.6 Repeated START Support

Enables combined transfers (e.g., register write → read):

- WRITE → RESTART → READ
- Prevents releasing the bus
- Fully supported in SDR FSM

2.7 START / STOP Generation

START:

- SDA: High → Low (SCL High)

STOP:

- SDA: Low → High (SCL High)

Generated by dedicated START/STOP generator with timing guarantees.

2.8 ACK, NACK & T-Bit Handling

- ACK sampled on 9th clock
- NACK triggers STOP + error log
- T-bit handled during read continuation

2.9 Dynamic Address Assignment (ENTDAA)

Full support for DAA:

Steps:

1. Controller sends ENTDAA
2. Devices respond with 48-bit Provisional ID
3. Open-drain arbitration selects winning device
4. Controller assigns 7-bit dynamic address
5. Process repeats until no more responders

Hardware blocks:

- 64-bit shift register
- Arbitration logic
- Dynamic address counter
- Device table (minimum 8 entries)

2.10 Supported CCCs

Version 1 implements:

- **ENTDAA** — Dynamic address assignment

CCC subsystem linked to CMD FSM.

2.11 Register Interface (Kept optional for V1)

Supports APB or AXI-Lite.

Registers:

- CONTROL
- STATUS
- TXFIFO_LEVEL
- RXFIFO_LEVEL
- CMD
- ADDR
- LENGTH
- TXDATA

- RXDATA

2.12 Error Handling

Controller detects:

- NACK on address
- NACK during data phase
- Arbitration loss
- FIFO underflow/overflow
- Bus busy / illegal conditions

All errors logged in STATUS register.

3. Global Interface

Signal	Direction	Description
clk	Input	System clock
rst_n	Input	Active-low synchronous reset

3.1 Reset Behaviour:

- SCL is **released high**
- SDA is **released high**
- All FSMs are in IDLE
- FIFOs are empty
- STATUS register bits are cleared
- Dynamic address register is cleared
- No bus activity is generated automatically

3.2 I3C Bus Interface

Signal	Direction	Description
SCL	Output	I3C clock generated by controller
SDA	Inout	Open-drain data signal

Bus Rules:

- SDA stable during SCL high
- START: SDA ↓ while SCL high
- STOP: SDA ↑ while SCL high
- SDA released during read phases

3.3 Register Interface Expectations

DV should assume **memory-mapped register model**.

Key behaviors:

- Registers sampled on cmd_start
- Registers locked during active transaction
- Writes while busy → ignored or error

REGISTER MAP

Offset	Name	Description
0x00	Reserved	Reserved for Master.
0x04	CONFIG	Configuration fields for things setup before block is activated
0x08	STATUS	Status for Master and Slave. Not all bits used if only Slave.
0x0C	CTRL	Control for active use (enable, resets, event gen)(e.g. interrupts to the Master)
0x10	INTSET	Interrupt Enable
0x14	INTCLR	Interrupt Disable
0x18	INTMASKED	Masked interrupt snapshot (STATUS & INTSET)
0x1C	ERRWARN	Error and warning status bits
0x2C	DATACTRL	FIFO control (flush, thresholds)
0x30	WDATAB	Write byte into TX path (TX FIFO)
0x34	WDATABE	Write Byte Data as End (to-bus) register
0x38	MWDATAH	Write Half-word Data (to-bus) register
0x3C	MWDATAHE	Write Half-word Data as End (to-bus) register
0x40	RDATAB	Read a byte from RX FIFO
0x48	RDATAH	Read Half-word Data (from-bus) register
0x54	WDATAB1	Byte-only Write Byte Data (to-bus) register
0x60	CAPABILITIES	Block capability bits (shows features enabled at HW build)
0x64	DYNADDR	Dynamic address once assigned
0xFFC	ID	Block ID and revision

4. Transaction-Level Behavior

4.1 Command Execution Model

Expected flow:

1. Program registers
2. Load TX FIFO (if write)
3. Assert cmd_start
4. Controller sets BUSY
5. FSM executes autonomously
6. DONE asserted
7. BUSY deasserted

4.2 SDR Write Transaction

Protocol sequence:

- START
- Address + W
- ACK check
- Data bytes from TX FIFO
- ACK after each byte
- STOP

4.3 SDR Read Transaction

Protocol sequence:

- START
- Address + R
- ACK check
- Data sampled from SDA
- RX FIFO write
- Controller ACK/NACK
- STOP

5. CCC Handling Expectations

5.1 Broadcast CCC

- Address = Broadcast
- No data returned
- No per-target ACK dependency

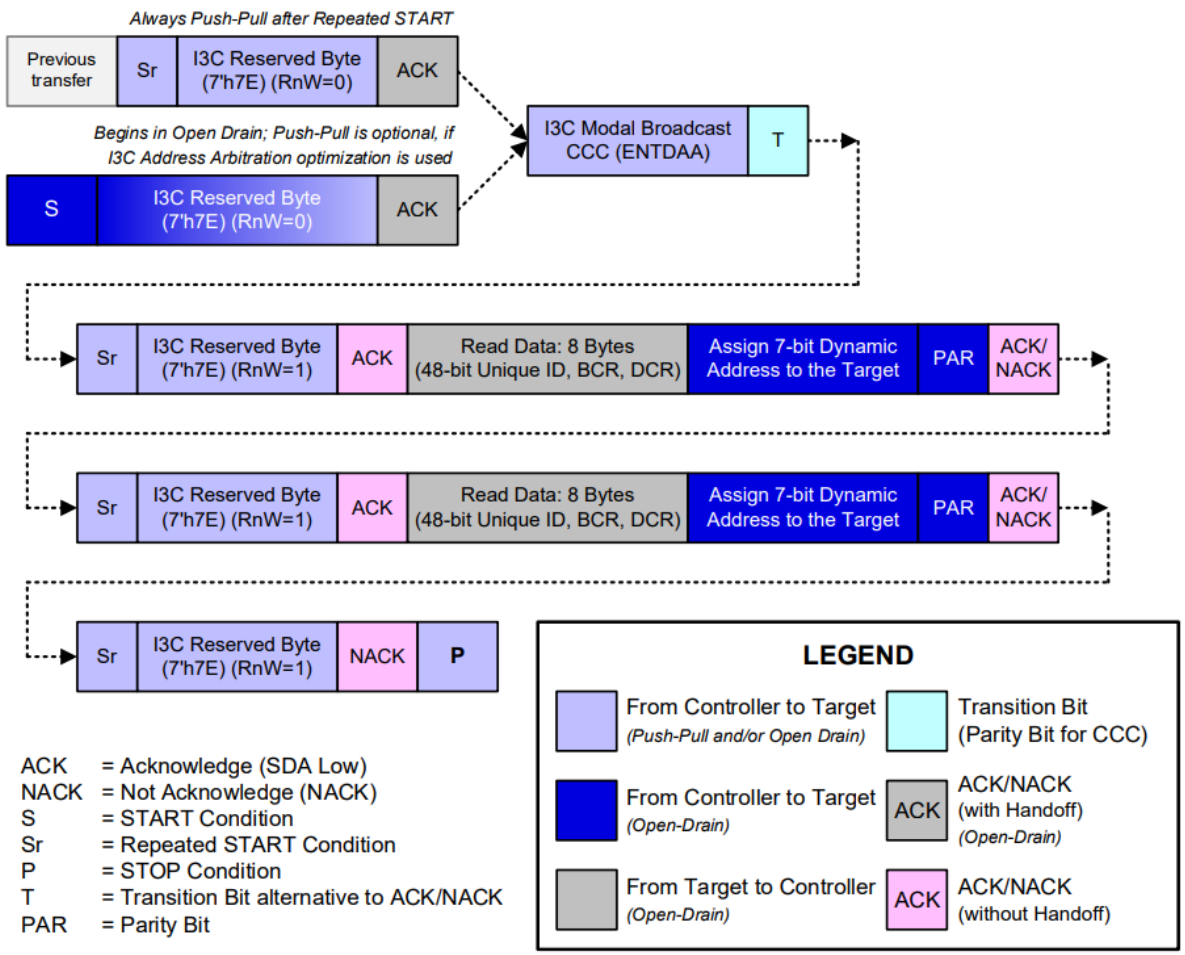
5.2 Directed CCC

- Addressed to single target
- Supports read/write CCCs
- ACK/NACK handled like private transfer

6. Dynamic Address Assignment (DAA)

6.1 DAA Entry Conditions

- Triggered only by explicit DAA command
- SDR FSM must be idle



6.2 DAA Flow

1. SEND 7E + W
2. SEND ENTDAA
3. Repeated START
4. SEND 7E + R
5. Arbitration on SDA
6. PID + BCR + DCR capture (64 bits)
7. Assign dynamic address with parity (odd)
8. Loop until no response

7. FSM Expectations

7.1 SDR FSM States

Supported operations

- START
- Address phase
- Multi-byte read/write

- ACK/NACK handling
- STOP

Key properties

- Uses **push-pull signaling** for I3C targets
- Uses **open-drain** for I²C targets
- Direction determined by dir
- Length determined by len

Error handling

- Address NACK
- Data NACK
- RX FIFO overflow

7.2 DAA FSM States

This FSM implements **ENTDAA exactly as specified**.

High-level flow

1. START
2. Broadcast 7E + W
3. Broadcast ENTDAA (0x07)
4. Repeated START
5. Broadcast 7E + R
6. **Release SDA**
7. Collect:
    - 48-bit PID
    - BCR
    - DCR
      using **open-drain arbitration**
8. Determine winning device
9. Transmit:
    - 7-bit dynamic address
    - Odd parity bit
10. ACK check
11. Repeat from step 4 until NACK
12. STOP

Arbitration details

- Arbitration occurs **bit-by-bit**
- Controller only drives SCL
- SDA is wired-AND from all targets
- Lowest concatenated {PID,BCR,DCR} wins

Outputs

- pid, bcr, dcr
- dyn_addr
- dt_en to store entry in device table

7.3. Bit Engine Contract

The Bit Engine is the **only block that drives SDA**.

Responsibilities

- Generate START and STOP conditions
- Shift 8-bit data MSB-first
- Sample SDA on SCL rising edge
- Drive SDA on SCL falling edge
- Generate ACK/NACK cycles
- Support **Repeated START**
- Support **ENTDAA arbitration mode**

Operating modes

Mode	arb_mode	Behavior
Normal	0	Byte-framed transfer with ACK
Arbitration	1	SDA released, bit-by-bit sampling

In arbitration mode:

- No ACK/NACK cycles
- No byte framing
- SDA is **always High-Z**
- Used exclusively by ENTDAA

This directly matches MIPI I3C spec §4.3.7.3.4.

8. FIFO Behavior

8.1 TX FIFO

- Read only during SEND_WR
- Underflow → ERROR + STOP

8.2 RX FIFO

- Write only during WAIT_RD
- Overflow → ERROR reported

9. SCL Generator

Generates the I3C SDR clock.

Features

- Programmable frequency (≤ 12.5 MHz)
- Push-pull or open-drain behaviour
- START / STOP handshake
- Bus idle enforcement 2us after STOP

SCL is **always controller-driven**.

10. Error Reporting Rules

Error	Expected Behavior
Address NACK	STOP + ERROR + DONE
Data NACK	STOP + ERROR + DONE
TX Underflow	STOP + ERROR + DONE
RX Overflow	ERROR flag set

All errors MUST:

- Terminate transaction
- Return controller to IDLE